

Hands-on Robot Control

4th Doctoral Summer School on Robotics and Intelligent Machines – DRIMS

Gold Sponsor



Silver Sponsor



Bronze Sponsor



Aug. 30 – Sept. 6, 2026
Poggio all'Agnello (Italy)



About



Marco Faroni
Politecnico di Milano



Michele Ferrari
CNR-STIIMA



Andrea Pupa
Univ. Modena e Reggio Emilia



Samuele Sandrini
CNR-STIIMA

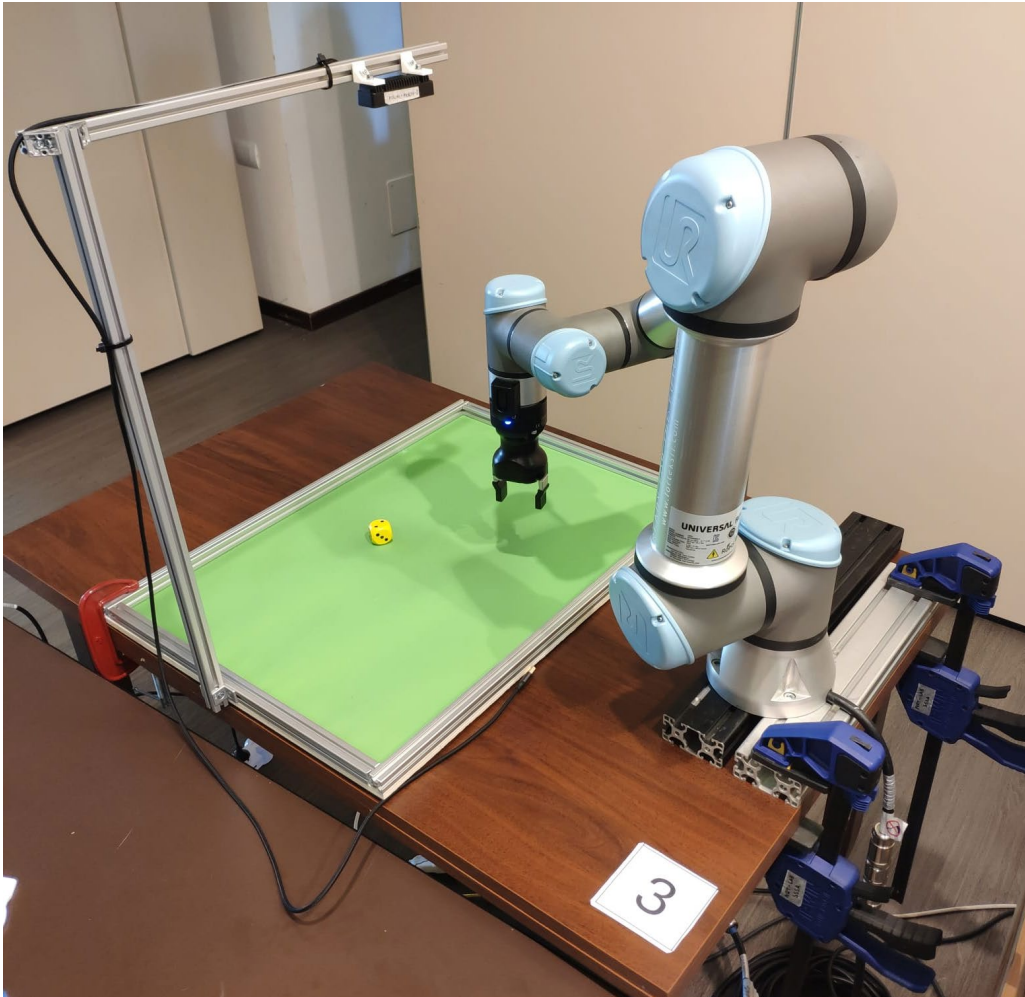


Roberto Fausti
CNR-STIIMA



Anna Bicchi
Politecnico di Milano

Hands on the DRIMS challenge



- Use the robot to move the die to expose the requested face
- Do it as fast as possible

Minimal skills:

1. Use camera to localize the die and identify the current face
2. Move the robot and actuate the gripper to grasp the die and re-orient it

Constraints: robot speed cannot be increased (for safety reasons)

The DRIMS codebase

[drims_summerschool](#) is a hub repo that contains three repositories to spawn and control the robots:

- [drims_cells](#): Contains robot descriptions, controllers, and launch files for setting up simulations and/or establishing communication with real robots.
- [easy_motion](#): Provides APIs, ROS actions, and Behavior Tree nodes for basic robot movements.
- [drims_dice_simulator](#): Provides a lightweight simulator for a dice.

These packages are already installed in the docker image.
You don't need to download them again.

Your code

Fork [drims homework](#) from Github



```
cd drims_ws/src
```

```
git clone https://github.com/<your github user name>/drims_homework.git
```

Task 1: Spawn a robot

Terminal 1:

```
sh start.sh
```

```
ros2 launch drims_description ur5e_<cell_number>_start.launch.py fake:=true
```

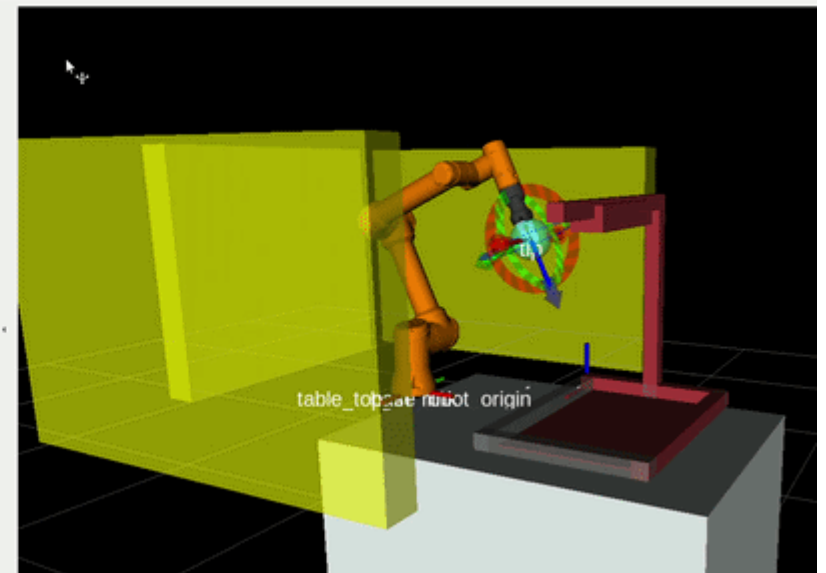
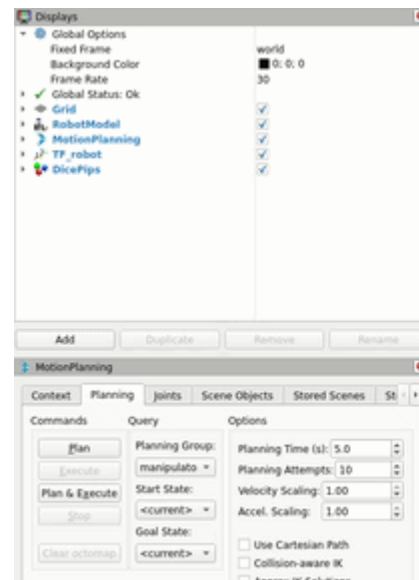
```
try to move it from RViz
```

Terminal 2:

```
sh connect.sh
```

```
ros2 topic echo /joint_states
```

Select a value between 1 and 4 inclusive.
Each cell's dimensions correspond to the physical setup.



Task 2: Run the demo

Robot Cell

Terminal 1: [Spawn the robot](#)

```
ros2 launch drims_description ur5e_1_start.launch.py fake:=true
```

Die Simulator

Will be replaced by your vision algorithm

First select the correct cell in the *drims_dice_simulator/config/dice_spawner_parameters.yaml* file.

Terminal 2: [Spawn simulated die](#)

```
ros2 launch drims_dice_simulator spawn_dice.launch.py face_up:=5 position:="[0.6, 0.2, 0.0]" surface_height=0.0
```

Details on the CLI arguments can be found in the repo README.

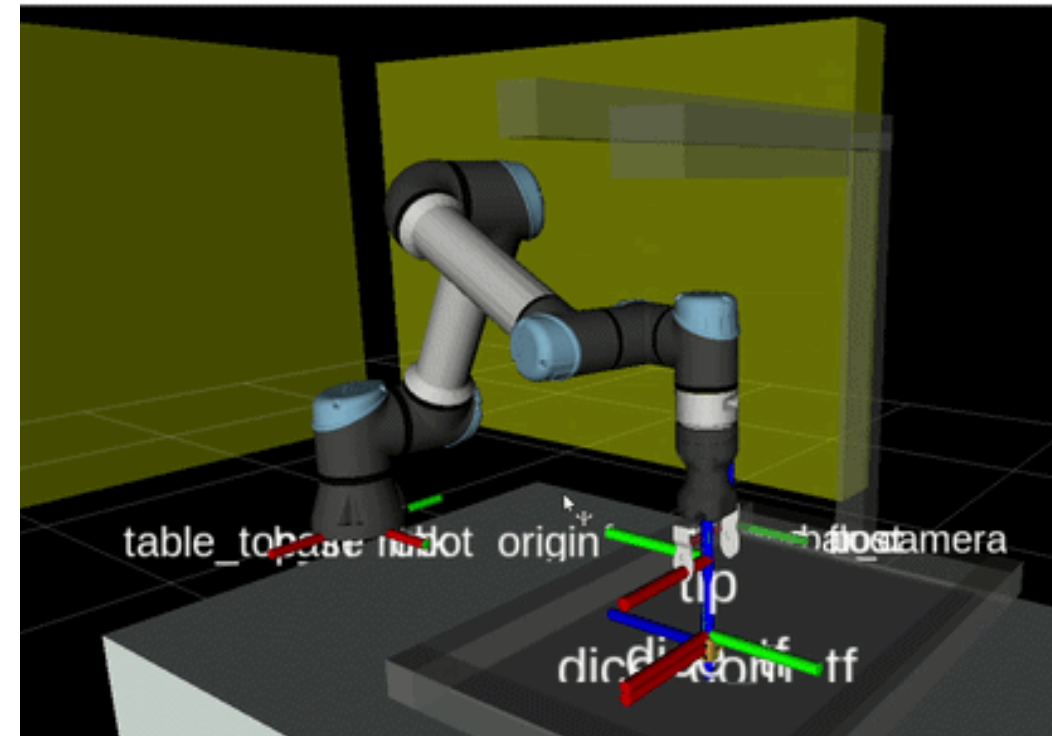
Task 2: Run the demo

Terminal 3: [Run example](#)

ros2 action list

ros2 launch drims_homework demo_node_start.launch.py

Inspect available actions and services!!



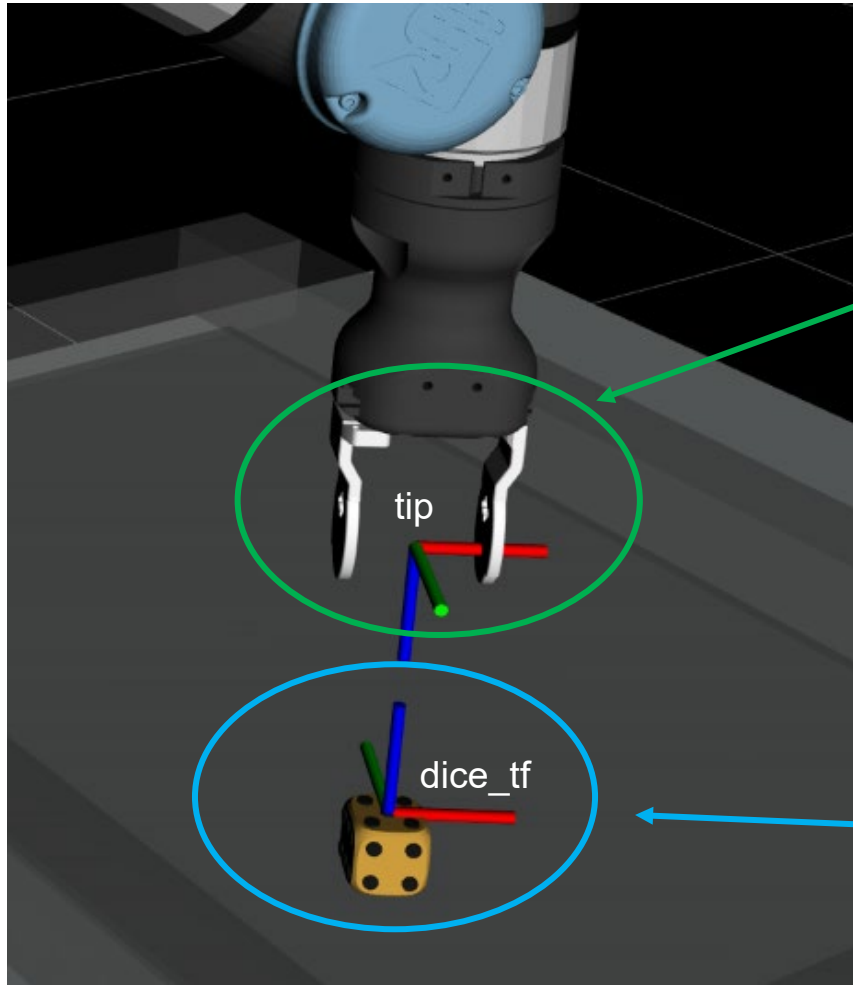
What you should do

State Machine

1. Create a new node* in [drims_homework](#) (use demo_node.py as an example)
2. Create a new launcher to launch the new node (see demo_node_start_launch.py)
3. The new node should:
 - Import the MotionClient library
 - Use MotionClient's methods to manipulate the die
 - Implement the logic flow to expose the correct face
4. Test it with different initial values of face_up and die positions

* need to update setup.py as well

Reason about TF



Robot's tip

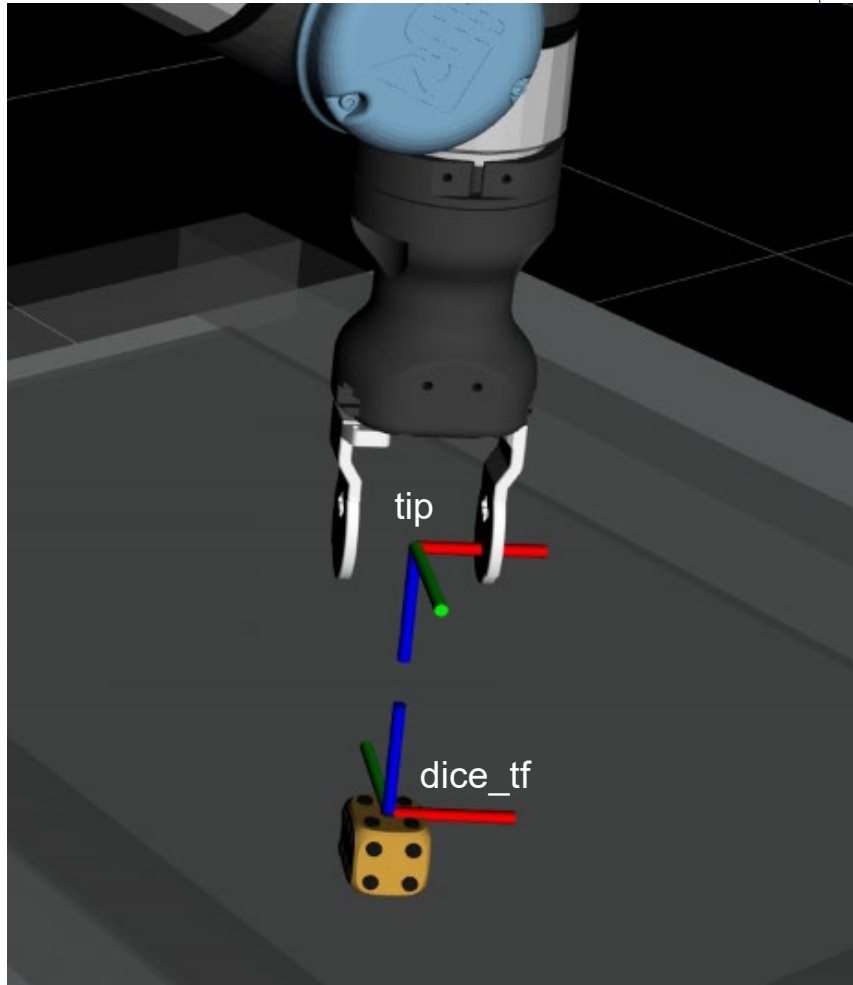
Dice tf

The vision node* will publish the die pose.

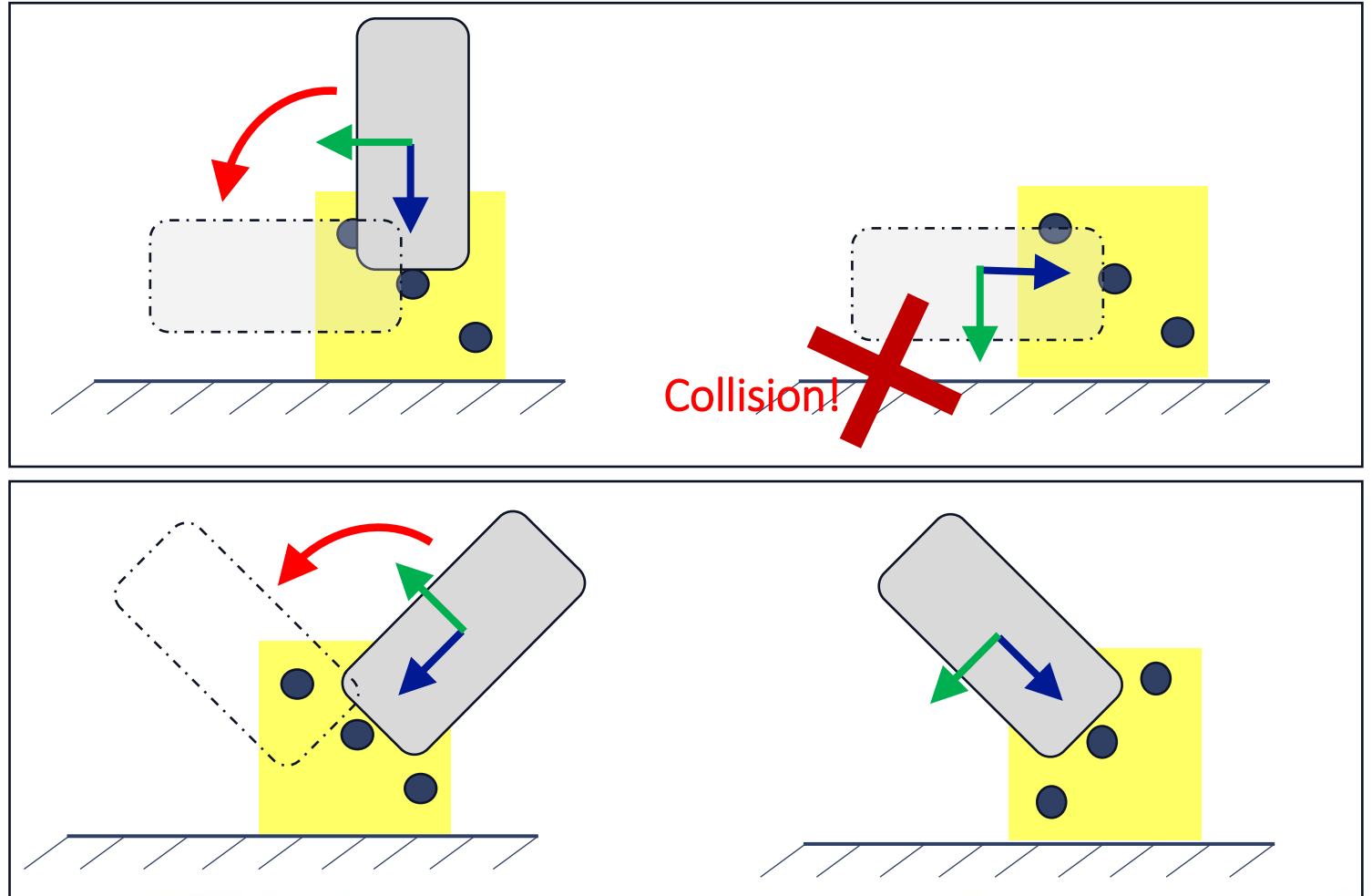
You need such a pose w.r.t. the robot tip so that you can move the robot above the die.

* In simulation you have the dice_simulator pkg.

Reason about TF



BE CAREFUL WITH ROTATIONS!



easy_motion: motion abstraction layer

You should directly use *easy_motion*'s APIs to move the robot.

From python:

- `move_to_joint(...)`
- `move_to_pose(...)`
- `plan_to_joint(...)`
- `plan_to_pose(...)`
- `get_ik(...)`
- `get_fk(...)`
- `gripper_command(...)`
- ... see https://cnr-stiima-iras.github.io/easy_motion/easy_motion/easy_motion.motion_client.html

easy_motion: motion abstraction layer

```
move_to_pose(pose: geometry_msgs.msg._pose_stamped.PoseStamped, cartesian_motion: bool = False,  
relative_motion=False, velocity_scaling: float = 1.0) →  
moveit_msgs.msg._move_it_error_codes.MoveItErrorCodes
```

Move the robot to a target pose.

Args:

pose (PoseStamped): Target pose for the robot. cartesian_motion (bool, optional): If True, uses Cartesian trajectories. Defaults to False. relative_motion (bool, optional): If True, consider pose as a displacement from the robot initial state. Defaults to False. velocity_scaling (float): Manual velocity scaling applied to the planned trajectory.

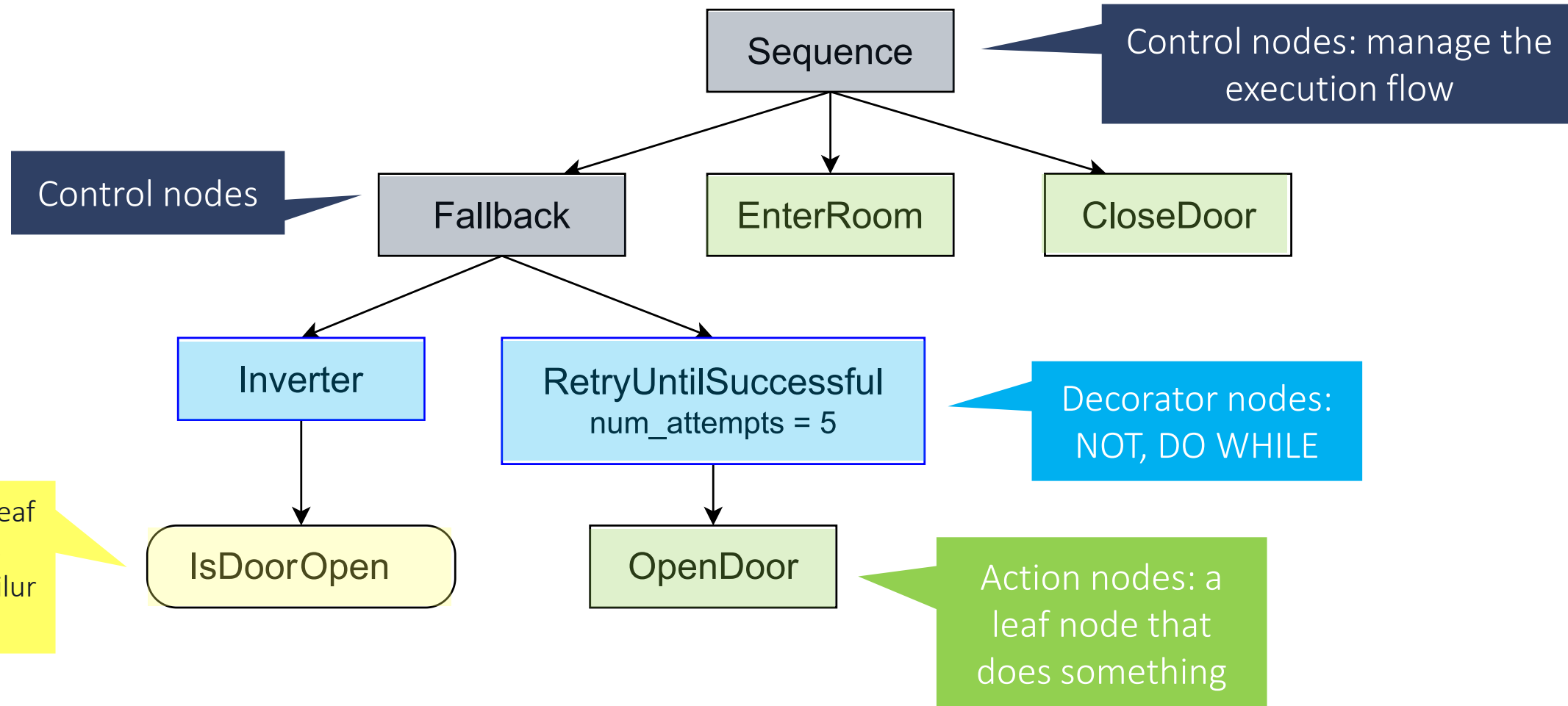
Returns:

MoveItErrorCodes: Result code returned by the motion planner.

Raises:

RuntimeError: If the action server is not available or the goal was rejected.

Optional: Behavior Trees



Optional: Behavior Trees

Terminal 1: [Spawn the robot](#)

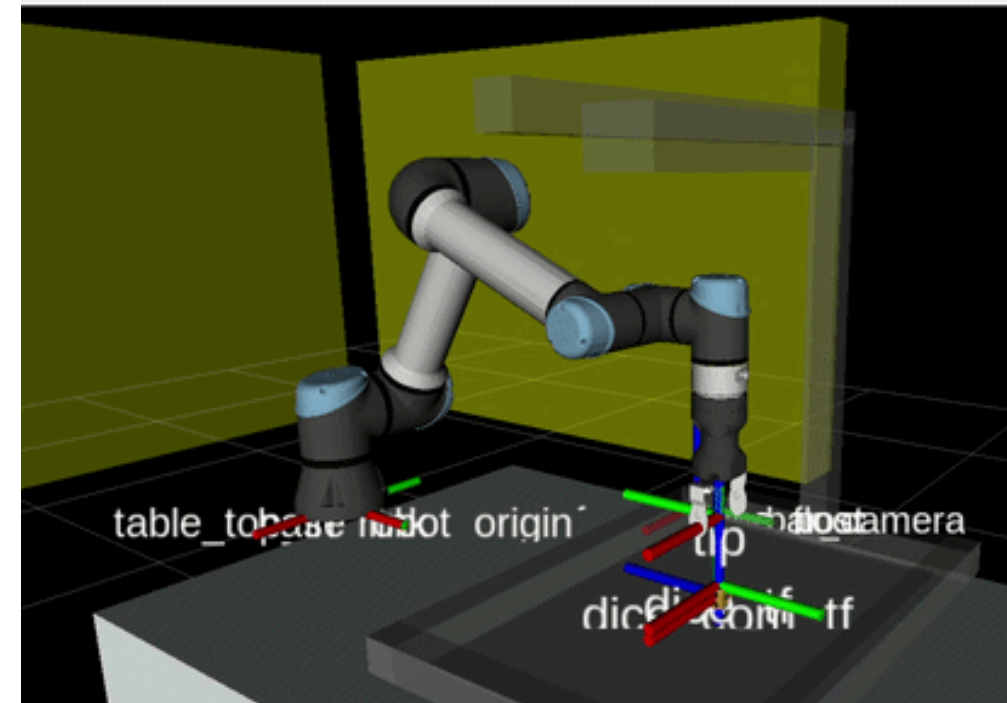
```
ros2 launch drims_description ur5e_1_start.launch.py fake:=true
```

Terminal 2: [Spawn simulated dice](#)

```
ros2 launch drims_dice_simulator spawn_dice.launch.py face_up:=5 position:=[-0.1, 0.0, 0.85]
```

Terminal 3: [Run example](#)

```
ros2 launch drims_homework demo_behavior_tree_start.launch.py
```



Optional: Behavior Trees

BEHAVIOR-TREE APPROACH

1. Create a new BT in `drims_homework` (use [trees/example.xml](#) as an example)
2. Modify `demo_behavior_tree_start.launch.py` to launch your BT
3. The new BT should implement the logic to expose the correct face:
 - You need to use control and decorator nodes to implement the logic
 - Action nodes directly call ROS2 actions. The name of the action and the parameters are specified in the xml

Summary of ToDos

1. Task 1: Spawn a robot and inspect topics and services
2. Task 2: Run the demo to pick the die
3. Task 3: Create a state machine to manipulate the cube and expose the desired face
4. Optional: Implement the same logic with Behavior Trees

Useful Stuff

4th Doctoral Summer School on Robotics and Intelligent Machines – DRIMS

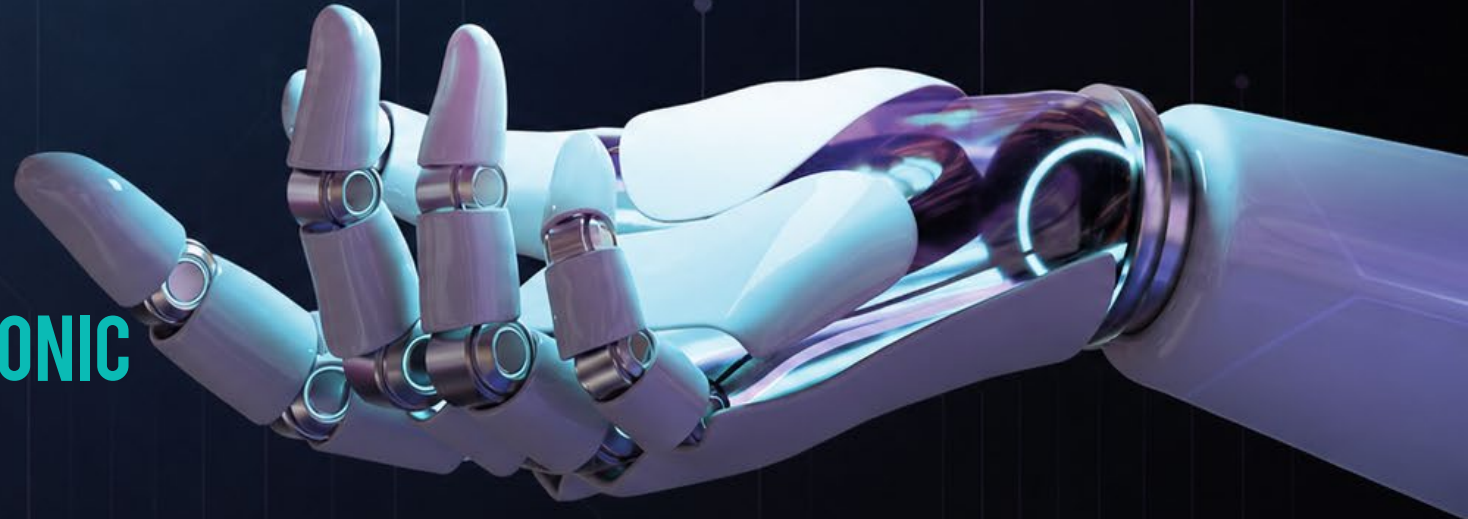
Gold Sponsor



Silver Sponsor



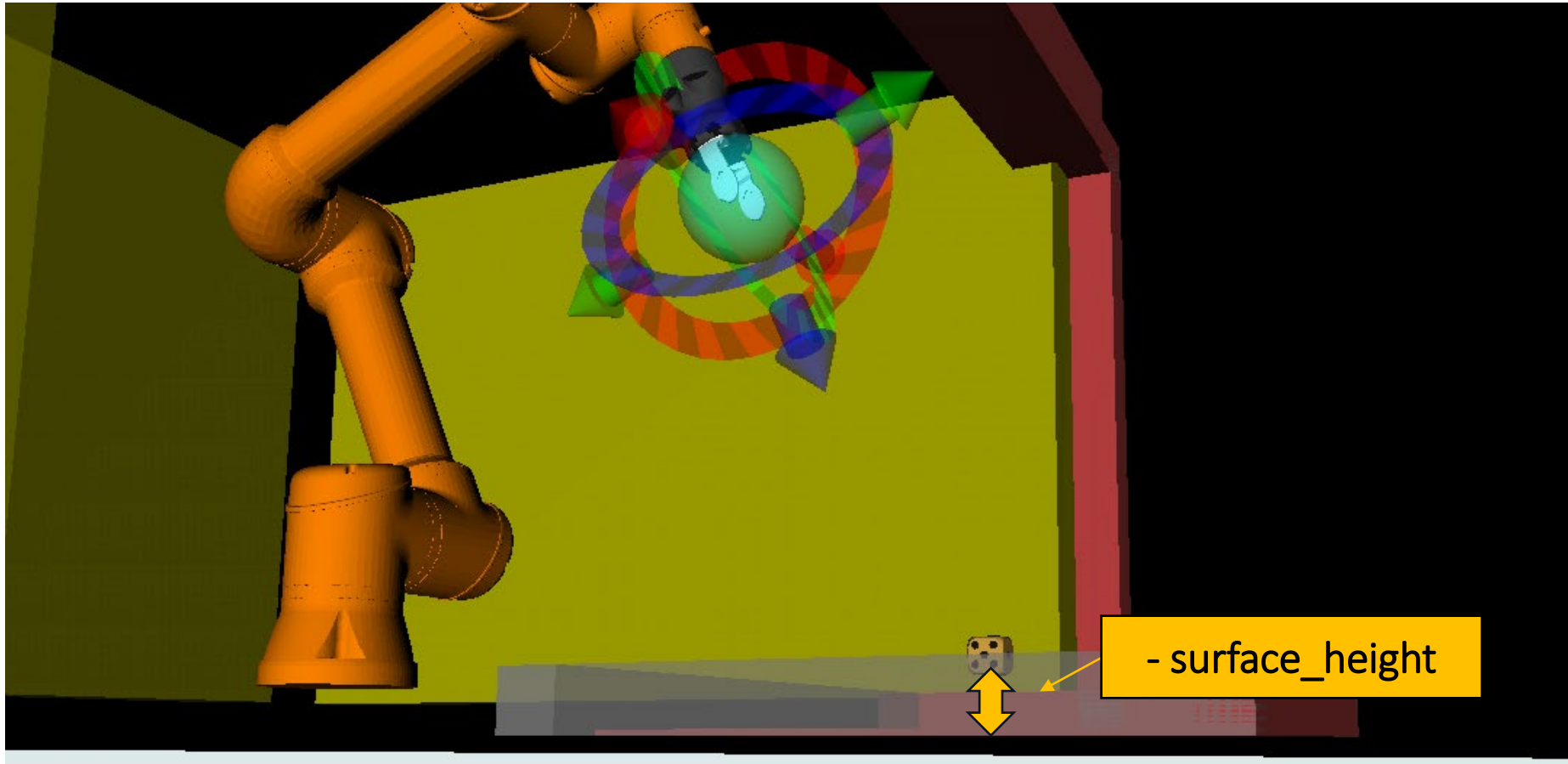
Bronze Sponsor



Aug. 30 – Sept. 6, 2026
Poggio all'Agnello (Italy)



Spawning the die at the correct height



Robot 1: -0.04
Robot 2: -0.02
Robot 3: -0.01
Robot 4: -0.02

This is wrt base_link

`ros2 launch drims_dice_simulator spawn_dice.launch.py face_up:=5 position:="[0.6, 0.2, 0.0]" surface_height=0.0`

How to Control the Standalone Gripper

In the 3D printing room, there is a **robotiq 85f gripper** mounting **touch sensors**.

How to use them:

1. **IMPORTANT:** Unmount the touch sensors to avoid breaking them during activation
2. Plug in the USB and connect to the DRIMS docker container
3. Terminal 1: [Launch the Robotiq driver](#)

```
ros2 launch robotiq_description robotiq_control.launch.py
```

Now you can mount the sensors again.

Terminal 2: [Call the action to move the gripper](#)

```
ros2 action send_goal /robotiq_gripper_controller/gripper_cmd  
control_msgs/action/GripperCommand "command: position: 0.4 max_effort: 0.0"
```

Full-close position is around 0.7.
Leave max_effort set to 0.0!

